



Capstone Proposal

Name: Lily McClellan

Capstone Title: **Causerie**

Project Summary

The term 'causerie' refers to an intellectual informal discussion. Often employed in language learning settings, it's often used for encouraging low-stress practice of a target language. That is exactly what this project will provide. Causerie is an AI-driven dialogue application packed with research-backed features aimed at allowing safe and controlled studying. Key features of this application include topic-based dialogues, a sticky note notation system, learning goals, targeted corrections/practice, and a reminder system for building study habits. Visually, this app will have a soft and welcoming UI and a simple mascot to give a personable aspect to dialogues.

Causerie will be a local desktop application aimed at portability and cross-platform compatibility, primarily supporting external AI API integration. The tech stack is as follows:

- Tauri2 (Software Framework)
- React/TS/Vite (Frontend)
- PIXIJS (Character Rendering)
- XState (State Machine)
- SQLite (Database)
- Rust (Backend)

The biggest anticipated challenge is coordinating deterministic application states across declarative React UI, imperative PIXIJS rendering, asynchronous LLM operations, persistent application state stored in SQLite, and the Rust application layer. LLM context orchestration will be paramount to the project's functionality, and will also present a valid challenge. In addition to this technical challenge, I am ensuring that all features align with linguistic research, from engaging direct/indirect learning strategies

to cultivating the correct environment to both encourage and challenge learners. While prior courses focused on simple asynchronous actions with linear state progression, Causerie extends these concepts to a more industry-appropriate level.

Motivation

I have a strong passion for linguistics, having studied 5+ foreign languages over my life (though I only speak English and French effectively). One of the most effective ways to learn a language is to *use it*, but like many others, I find those opportunities limited and stagnating. I hope to make a project that gives intermediate and advanced learners who are caught in 'learning limbo' a place to sharpen and practice skills. Technically, Causerie will provide me with vital full-stack experience and an opportunity to create a product that addresses a genuine problem experienced by language learners. With the complex interlocking pieces, I will have the opportunity to work on a substantially different scale from projects I have previously completed. Ultimately, I feel Causerie is a perfect professional demonstration of and challenge to my repertoire of skills.

Technologies and Research Requirements

Technologies

- **Tauri2** – Tauri2 will be used as the native desktop shell which the application is built upon. It allows for the use of standard web technologies while allowing privileged operations behind the Rust layer.
 - I do not have experience with Tauri, but I have some experience with a similar technology: Electron. Tauri was chosen over Electron for its lower memory footprint, which is beneficial for applications that are meant to be open for extended periods of time. I plan to learn Tauri through the official documentation, videos, and demo projects.
- **React** – React is the main framework for developing the frontend of the application, which was chosen for its versatility in developing dynamic interfaces as well as its place as a standard choice in industry-level projects.
 - I am familiar with React, having used it in other projects, but will be taking my skills far further than I had before in tying React modules/components to the state machine.
- **TypeScript** – TypeScript serves as the frontend scripting language for developing React components. It is a natural choice for the project, having all the benefits of being an integrated web technology while retaining type safety. TypeScript will also be responsible for communication between React/Vite and PixiJS/XState.
 - I have a lot of experience in using TypeScript, but will need to do additional research for developing React components.

- **Vite** – Vite is the frontend build tooling system, managing development builds and production deployment.
 - I have limited experience with Vite, and will likely need to demo some projects to ensure I fully understand the Vite ecosystem.
- **PixiJS** – PixiJS is an HTML5 creation engine specialized in 2D rendering through WebGL. This component is for rendering the mascot of the application, which is necessary due to its complex visual and programmatic states.
 - I have never used PixiJS before nor WebGL, and will be learning this technology from scratch. I plan to learn it via official documentation, example projects, and online videos.
- **XState** – XState is a state management solution that naturally integrates with TypeScript. XState provides an event-driven approach to handling state changes across the frontend components.
 - I have never used XState, and have never used complex state management solutions before, but have a strong understanding of its use cases and development cycle. I will develop my knowledge of this technology via official documentation, example projects, and online videos.
- **SQLite** – SQLite is a lightweight database built into most platforms/frameworks. It will be used for local data storage for handling persistence of states, user preferences, progress tracking, etc.
 - I have extensive experience with SQLite and adjacent technologies, and I am very confident in my ability to develop with it.
- **Rust** – Rust will serve as the backend language for creating the main business logic. This includes API calls, system-level state management, and behind-the-scenes application logic.
 - I have worked with Rust before, but will likely need to revisit my old projects to remember syntax and do more research into the borrow checker.
- **OpenAI API** – The primary AI API that the application will be developed to use. This API will primarily power the AI conversations, with careful guards and monitors in place to detect and mitigate inaccurate or inappropriate responses.
 - I am not familiar with direct API calls, and will need to research various technicalities such as I/O structure, context/token management, and failure handling.
 - In addition to the API usage, I will need to do deeper research into safeguarding the AI responses to ensure that they are accurate and appropriate to the defined parameters of a given conversation. This will also include safeguarding against bogus inputs that may produce unexpected results.
- **WiktApi** – WiktApi is a multilingual dictionary API, which will be used for translation related functions in the application.
 - I will need to familiarize myself with structuring API calls, but will otherwise have no issues using this API.

Tools

- **WebStorm** – WebStorm is a JetBrains IDE specialized in web-based development, making it an excellent choice for developing the frontend of the application. WebStorm does not directly support PixiJS or XState, but there are community plugins available to assist with this.
 - I have used WebStorm extensively and will only need to research the intricacies of integrating PixiJS and XState.

- **RustRover** – RustRover is a JetBrains IDE specialized in developing Rust projects. I will use it for developing the backend of the application, and will be using built-in testing features to ensure the backend and API side functions as expected.
 - I am sufficiently familiar with RustRover and do not anticipate additional research requirements.
- **Codex** – Codex will serve as a development assistant for code review, debugging, and implementation support.
 - I am sufficiently familiar with Codex and do not anticipate additional research requirements. Final implementation, architectural decisions, and art will all remain part of my Capstone development process outside of AI usage.
- **Aseprite** – Aseprite is a digital art software specifically targeted at pixel art, which will be the primary style of the mascot for the project. This software may not be used if I choose to source my art externally.
 - I am familiar with Aseprite and pixel art and will not need to do any additional research unless I choose to source the art instead.

Platforms

- **Linux** – Causerie will primarily be developed on Linux, thus Linux will serve as the primary development and preference platform.
 - I am highly familiar with Linux and do not anticipate significant compatibility research outside OS-level notifications.
- **Windows/macOS** – Causerie will not be developed on these platforms, but they will be involved in the testing process to ensure expected cross-platform functionality.
 - I will need to do additional research in my aforementioned technologies to ensure that I have cross-platform compatible functions and/or cross-platform fallbacks. I will also need to research OS-level notifications for my notification system.

Research

- **Learning Strategies/Second Language Acquisition Strategies** – Causerie will be built around direct and indirect learning strategies (which more directly expands into Second Language Acquisition Strategies: cognitive, metacognitive, and social/affective) to help learners more effectively learn their target language.
 - I have done extensive research into learning strategies, and have already mapped out each feature with these strategies in mind. For example:
 - Stickies - DIRECT STRATEGIES - Utilises cognitive strategies of mental links, imagery, and mnemonics.
 - Mascot - INDIRECT STRATEGIES - Utilises affective strategies of reducing anxiety and making a positive emotional impact on learning. Utilises social strategies of collaborating and empathising.
 - And so on.
 - Many metacognitive strategies will not be included (alongside their respective features) in the MVP. These include:
 - Evaluating user progress - Proficiency tests
 - Monitoring user progress - Testing routine and proficiency score
 - Planning user progress - Setting incremental S.M.A.R.T. goals

- **Proficiency Monitoring (Stretch)** – Causerie will follow the Common European Framework of Reference for Languages (CEFR). Non-European languages are outside the MVP scope. This divides proficiency into 6 standard levels: A1 (beginner), A2, B1 (intermediate), B2, C1 (advanced), C2. CEFR also has a standard for assessing language comprehension, which will be utilised in the proficiency monitoring stretch goal.
 - I am deeply familiar with the CEFR system, but will need to do targeted research into how to properly evaluate one's proficiency level before I can implement the proficiency monitoring system.

Minimum Viable Product (MVP)

Initial Set-Up

- On initial launch, the application should run an initial set-up. Initial set-up should have the user select a target language, and proficiency level (including 'unknown')

Note: Preferred grammatical gender/form of address is necessary to many target languages, which involve gendered language.

- After initial launch, the application should run an introductory conversation about introducing yourself. This asks for the user's name, gender, and preferred areas of practice. This will be programmed, and will not involve the AI chatbot.
- The application should (after set-up) allow the user to create threads and give them a name and topic.

Threads

- Threads should each have their own stored contexts to guide the chatbot in responses (alongside conversation history).
- Creating a thread will open up a thread workspace. This includes a section for the mascot, a section for the conversation history, and a section for the user to enter responses.

Character/Mascot

- The mascot will minimally change states between greeting (initial prompt), idling (time without prompt), thinking (awaiting API response), and talking (outputting API response)
- Mascot should have an artistic representation for the aforementioned states and be able to transition between them as states update.

Sticky Notes

- During conversations, the user can highlight (a) word/s and select a sticky option, which will add a sticky note to the highlighted section.
- Sticky notes colours can be selected on creation/edit. Stickies appear as a clickable highlight overlaid on words wherever they appear in chatbot responses.
- Sticky notes can store a title (shown when hovering over a stickied phrase) and a body (shown when clicking on a highlight).
- Stickies can be deleted via button click.
- Stickies will have at least one placeholder value (chosen at random if multiple) showing the user what an effective note looks like (engaging cognitive strategy– NOT writing translations)

Goals

- User should be able to define up to three learning goals (such as “I want to expand my vocabulary” or “I want to work on my grammar”) which the AI will focus on when providing feedback
- Goals should minimally be selectable from a list.

Technical

- The application can save and update user preferences.
- The application will fully bundle and launch on Linux
- The application will ask for an OpenAI API key (MVP only supports this AI method).
- The application will minimally manage LLM context/tokens/conversations to ensure modular verifications and checks without context bleeding.
- The application will gracefully handle/display errors
- The application will be able to query WikitApi for multilingual dictionary lookups and lexical references

Styling

- The application should minimally have a consistent dark theme
- Application UI should be consistent across pages

Secondary Features

Initial Set-Up

- The application should ask for frequency of reminders (options for what time of day and if daily/what days of the week).

Threads

- Threads can either be initialised with a user prompt or a generated prompt from the AI.

Hints

- During conversation, a user can highlight (a) word/s and select a hint option, which opens up a menu that shows the dictionary definitions of each of the highlighted words.
- During conversation, a user can highlight (a) word/s and select a translate option, which prompts the mascot to provide a brief translation of the section (not recorded to conversation history)

Reminders

- The application should send push notification reminders at the defined dates/times.
- Push notification reminder should include an encouraging remark and on click should open the application.
- The application should register notification schedules as OS-level notifications to ensure they are delivered regardless of the app state.

Character/Mascot

- The mascot should provide user-friendly error feedback for common errors (API cannot be reached, API limit reached, internal error, etc)

Technical

- The application can be packaged, installed, launched, and have its entire MVP workflow functional on Linux, Windows, and macOS
 - Cross-platform testing will be done throughout development

Stretch Goals

Progress

- The application should provide proficiency tests to gauge your proficiency in your target language
- The application should be able to show you your proficiency over time
- The application should provide targeted feedback based on proficiency scores

Character/Mascot

- The mascot should be animated and have more sprites/expressions depending on the conversation
- The mascot should have 'Shimeji Software'-like interactable actions within the application (primarily moving around and having a clickable interaction menu)

Goals

- Goals should be dynamically set by the user and encourage S.M.A.R.T. goal setting strategies.

Technical

- User can choose between different API providers (DeepSeek, Claude, etc) or local models (Ollama)
- The application should maturely handle context, managing learner configuration, thread context, goals, conversation state, and auxiliary operations.

Styling

- The application should have multiple colour schemes
- Application UI interactions should have coherent and consistent animations

Milestones and Timeline

Note: Non-critical bugs will be documented throughout development and prioritised for the dedicated refinement period in Week 8.

Week 0 - Pre-Development - N/A

- Begin technical research (platforms, tools, etc)
- Finalise non-technical research (CEFR, Art Sourcing, etc)
- Finalise planning
- Create documentation for Causerie (API calls, data storage, etc)
- Begin creating/sourcing mascot art

Week 1 - Prototyping - 20h+

- Create styling/UI/UX mock-ups
- Create/Source mascot sprites
- Create AI API prototype tests
- Create dictionary API prototype tests
- Create mascot rendering/state demo
- Create an end-to-end application vertical slice
 - Slice should be a very rough proof of concept - sending a message from a react interface, updating an XState transition, Tauri/Rust boundary, Rust logic, OpenAI request, returned response, state update, PixiJS mascot response, and SQLite persistence

Week 2 - Initial Development - 20h+

- Establish Tauri/React/Rust application
- Establish XState ownership boundaries (between PixiJS, React, and AI API)
- Integrate basic PixiJS
- Initialise SQLite DB (schema, DAOs, and DAL)

Week 3 & 4 - Frontend Core Development - 40h+

- Build conversation workspace
- Add state-driven functionality to mascot
- Build Threads feature UI
- Build application onboarding
- Implement sticky-note interactions
- Develop application configuration menu
- Test reproducible visual design on Windows/Mac

Week 5 & 6 - Backend Core Development - 40h+

- Implement mature context management (with frontend config)
- Add application persistence
- Add goals (MVP version)
- Add sticky persistence
- Add OpenAI API calls
- Add threads API request/response flow
- Add threads persistence
- Add error handling (APIs, user input, PixiJS, XState, and SQLite)
- Test application bundling for Windows/Mac

Week 7 - MVP Completion and Secondary Feature Development - 20h+

MVP

- Develop and apply MVP styling
- Add WziktAPI calls
- Implement remaining MVP integration/errors

Secondary Development - Prioritised as time permits

- Implement hint interactions (with hint API response)
- Add AI-started threads as an option on creating threads
- Route error response to mascot responses
- Add reminder configuration
- Add cross-platform OS-level reminders

Week 8 - Refinement Stage - 20h+

- Polish frontend styling
- Cleanup UX (remove demo elements, add improvements, and polish final design)
- End-to-end functionality testing
- Dedicated bug fixing/stabilisation
- If completed early, begin week 9

Week 9 - Stretch Goals and Presentation Design - 20h+

- Implement stretch goals as time permits

(All items are optional and pick-and-choose by preference)

- Progress/Metacognitive System
 - Add proficiency test
 - Add proficiency tracking
 - Add targeted feedback from proficiency tests
- Mascot Overhaul
 - Add mascot animations
 - Add additional mascot sprites/states
 - Add mascot movement
 - Add mascot click-menu
- Goals Overhaul
 - Enable custom user goals with S.M.A.R.T. goal assistance
- AI API Overhaul
 - Add functionality for other AI APIs
 - Add more mature context ecosystem
- Styling Overhaul
 - Add multiple colour schemes/themes

- Add/polish UI animations
- Create capstone presentation
- Rehearse capstone presentation

Week 10 - Presentation - 20h+

- Finalise capstone development
- Finalise capstone presentation
- Perform final testing
- Present capstone

Faculty Review

Proposal Approved: ☐ Yes ☐ No

Faculty Comments: